

Python · NumPy · pandas · scikit-learn Cheat Sheet

NumPy Basics

```
import numpy as np
a = np.array([1, 2, 3])
a.shape, a.dtype, a.ndim
np.zeros((3,4)); np.ones(5)
a.mean(), a.std(), a.sum()
a[0:2]          # slicing
a[a > 1]        # boolean mask
np.sqrt(np.mean(a**2)) # RMS
```

pandas Basics

```
import pandas as pd
df = pd.read_csv("file.csv")
df.head(); df.info(); df.describe()
df["col"]          # a column
df.iloc[0:5]       # row slice
df[df["label"]==1]  # filter
df.isna().sum()    # missing data
df.groupby('label').mean()
```

scikit-learn: the fit / predict Pattern (every model in Weeks 1–6 follows this)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=0.3)
scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train),
scaler.transform(X_test)

model = LogisticRegression() # or SVC(),
RandomForestClassifier(), ...
model.fit(X_train, y_train)
model.predict(X_test)
model.score(X_test, y_test) # accuracy
```

Simple Signal Features

```
mean    — window.mean()
std dev — window.std()
RMS     — sqrt(mean(window**2))
peak-to-peak — window.max()-window.min()
```

Common Gotchas

Always fit the scaler on TRAIN only, then transform test.
 random_state=0 → reproducible train/test splits.
 Check class balance before trusting accuracy.
 Shape mismatch? print(X.shape) is your first move.

Coming in Week 7: the PyTorch equivalent of this workflow — **tensor** → **model** → **loss.backward()** → **optimizer.step()**.